

```

/**
 * @file CommunicationService.cpp
 * @date 17 juil. 2018
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 *
 */

#include "CommunicationService.h"

namespace ecl
{
namespace sys
{
namespace xm
{

CommunicationService::CommunicationService()
{
}

CommunicationService::~CommunicationService()
{
}

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */

```

```
/**
 * @file CommunicationService.h
 * @date 17 juil. 2018
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 */

#ifndef ECL_SYS_XM_COMMUNICATIONSERVICE_H_
#define ECL_SYS_XM_COMMUNICATIONSERVICE_H_

#include <ecl/sys/CommunicationService.h>

namespace ecl
{
    namespace sys
    {
        namespace xm
        {

            class CommunicationService : public sys::CommunicationService
            {
            public:
                CommunicationService();
                virtual ~CommunicationService();

                virtual void createSamplingPort(InplacePolymorph<Port>& port, const char8_t* portName);
                virtual void createQueuingPort(InplacePolymorph<Port>& port, const char8_t* portName);
            };

        } /* namespace xm */
    } /* namespace sys */
} /* namespace ecl */

#endif /* ECL_SYS_XM_COMMUNICATIONSERVICE_H_ */
```

```

/**
 * @file
 * @date      $Date$
 * @version   =TAG=
 * @author    C-S Team
 * @copyright CNES (LVCUGEN project)
 *
 * @brief This module abstracts the hypervisor and the hardware interfaces to ease eventual future portages.
 *
 * It encapsulates the used hypervisor services and contains all the target specific code.
 * It provides services for :
 *   - initializing the HAL,
 *   - managing the partition scheduling:
 *     - get the partition execution slot id,
 *     - suspend the partition execution until its next execution slot,
 *     - halt the partition itself,
 *   - managing the following signals:
 *     - the shutdown signal raised to inform the partition of its upcoming halt,
 *     - the slot start signal raised at each partition execution slot start,
 *   - managing a one-shot timer based on the absolute time
 *   - managing the inter-partition communication through sampling and queuing ports,
 *   - managing memory access outside the one allocated to the partition.
 *
 * In addition, it provides services to read, write or read/write a 32b value in a atomic way (i.e. in one assembly instruction).
 * So that, it can't be interrupt even by the hypervisor.
 */
#ifdef __HAL__
#define __HAL__

#ifdef __cplusplus
extern "C" {
#endif

#include <bsp/types.h>
#include <bsp/misc.h>

/**
 * Define a invalid id of port
 */
#define HAL__C_INVALID_ID_PORT (-1)

/**
 * Forward declaration of the trap context structure.
 */
struct trapCtx;

/**
 * Type of read-only memory address.
 */
typedef const void* T_CONST_ADR;

/**
 * Type of the only parameter of a trap handler subroutine : it contains the interrupt call context (in particular, the trap number)
 */
typedef struct trapCtx HAL__T_HYPERVISOR_TRAP_CTX;

/**
 * Prototype of a trap handler subroutine
 */
typedef void (*HAL__T_HYPERVISOR_TRAP_HANDLER) (HAL__T_HYPERVISOR_TRAP_CTX*);

/**
 * List of the directions allowed for a port
 */
typedef enum _HAL__E_PORT_DIRECTION
{
    HAL__C_PORT_DIRECTION_SOURCE = 0x2,
    HAL__C_PORT_DIRECTION_DESTINATION = 0x1
} HAL__E_PORT_DIRECTION;

/**
 * List of the identifiers of the signals supported by the HAL.
 */
typedef enum _HAL__E_SOFT_IRQ_ID
{
    HAL__C_SOFT_IRQ_ID_SHUTDOWN = 3,
    HAL__C_SOFT_IRQ_ID_SLOT_START = 8,
    HAL__C_SOFT_IRQ_ID_TIMER = 0
} HAL__E_SOFT_IRQ_ID;

/**
 * List of the identifiers of the hard IRQs supported by the HAL.
 */
typedef enum _HAL__E_HARD_IRQ_ID
{
    HAL__C_SOFT_IRQ_ID_PPS = 4,
    HAL__C_SOFT_IRQ_ID_TF = 11
} HAL__E_HARD_IRQ_ID;

/**

```

```

* Informations about queuing ports.
*/
typedef struct _HAL__QUEUEING_PORT_INFO
{
    T_UINT32 maxMsgSize;
    T_UINT32 maxNoMsgs;
    HAL__E_PORT_DIRECTION direction;
} HAL__QUEUEING_PORT_INFO;

/**
* Informations about sampling ports.
*/
typedef struct _HAL__SAMPLING_PORT_INFO
{
    T_UINT64 refreshPeriod;
    T_UINT32 maxMsgSize;
    HAL__E_PORT_DIRECTION direction;
} HAL__SAMPLING_PORT_INFO;

/**
* Atomic read of a 32bits resource.
* @param[in] _read_adr the address to read
* @pre _read_adr: it should be a valid address of a T_UINT32 resource
* @return the value read at _read_adr address
*/
static MISC__C_MACRO_ATTRIBUTE_INL T_UINT32 HAL__AtomicRead(volatile T_UINT32 *_read_adr)
{
    T_UINT32 read_value = 0;

    __asm__ volatile(
        "LD [%[val_addr]],%[val_reg]"
        : [val_reg] "=&r" (read_value)
        : [val_addr] "r" (_read_adr)
    );

    return read_value;
}

/**
* Atomic write of a 32 bits resource.
* @param[in] _write_adr the address to write
* @param[in] value the value to write
* @pre _write_adr: it should be a valid address of a T_UINT32 resource
*/
static MISC__C_MACRO_ATTRIBUTE_INL void HAL__AtomicWrite(volatile T_UINT32 *_write_adr, T_UINT32 value)
{
    __asm__ volatile(
        "ST %[val_reg],[%[val_addr]]"
        :
        : [val_reg] "r" (value),
          [val_addr] "r" (_write_adr)
    );
}

/**
* Atomic read/write of a 32b resource.
* This function performs an atomic read/write operation on a 32b resource.
* The pointed resource is read then written with the new value, and the read value is returned.
* @param[in,out] _adr_value the address of the 32b resource.
* @param[in] value_to_write the value to write at _adr_value address after the reading
* @pre _adr_value: it should be a valid address of a T_UINT32 resource
* @return the value read at _adr_value address before the writing
*/
static MISC__C_MACRO_ATTRIBUTE_INL T_UINT32 HAL__AtomicReadWrite(volatile T_UINT32 *_adr_value, T_UINT32 value_to_write )
{
    /* swap performs a simultaneous load and store operation,
     * which effectively swaps the value in memory at swap_addr with the value in register swap_imm initialised with the value_to_wr
     * ite argument*/
    __asm__ volatile(
        ".align 16\n\t" "SWAP [%[swap_addr]],%[swap_imm]"
        : [swap_imm] "=&r" (value_to_write)
        : [swap_addr] "r" (_adr_value),
          "[swap_imm]" (value_to_write)
    );

    /* return the value read before the writing and contains now in value_to_write */
    return value_to_write;
}

extern void HAL__Init(HAL__T_HYPERVISOR_TRAP_HANDLER ptr_UnexpectedTrapHandler);

extern T_UINT32 HAL__GetSlotId(void);
extern T_UINT32 HAL__GetPartitionId(void);
extern void HAL__HaltMySelf(void);
extern void HAL__WaitNextSlot(void);

```

```
extern void HAL__SoftIrqSubscribe(HAL__E_SOFT_IRQ_ID soft_irq_id, HAL__T_HYPERVISOR_TRAP_HANDLER _handler_address);
extern void HAL__HardIrqSubscribe(HAL__E_HARD_IRQ_ID hard_irq_id, HAL__T_HYPERVISOR_TRAP_HANDLER _handler_address);
extern void HAL__SoftIrqAllow(HAL__E_SOFT_IRQ_ID soft_irq_id);
extern void HAL__HardIrqAllow(HAL__E_HARD_IRQ_ID hard_irq_id);
extern void HAL__IrqsDisable(void);
extern void HAL__IrqsEnable(void);

extern void HAL__GetTimeUs(T_UINT64 *_absolut_date_in_us);
extern void HAL__SetTimer(T_UINT64 timer_date_in_us);

extern T_UINT64 HAL__GetIrqLastOccurenceTime(T_UINT8 irq_number);

extern E_OK_KO HAL__GetSamplingPortInfo(const T_CHAR* _port_name, HAL__SAMPLING_PORT_INFO* _info);
extern E_OK_KO HAL__CreatePortSampling(const T_CHAR* _port_name, T_UINT32 max_msg_size, HAL__E_PORT_DIRECTION port_direction, T_UINT64 refresh_period_in_us, T_INT32* _port_desc_or_status);
extern E_OK_KO HAL__WriteInPortSampling(T_INT32 port_desc, T_CONST_ADR _adr_msg, T_UINT32 size_msg, T_INT32* _exec_status);
extern E_OK_KO HAL__ReadInPortSampling(T_INT32 port_desc, T_UINT32 max_size, T_ADR _adr_msg, E_BOOL* _validity, E_BOOL* _no_msg, T_INT32* _read_msg_size_or_status);

extern E_OK_KO HAL__GetQueuingPortInfo(const T_CHAR* _port_name, HAL__QUEUEING_PORT_INFO* _info);
extern E_OK_KO HAL__CreatePortQueuing(const T_CHAR* _port_name, T_UINT32 max_msg_size, HAL__E_PORT_DIRECTION port_direction, T_UINT32 depth, T_INT32* _port_desc_or_status);
extern E_OK_KO HAL__WriteInPortQueuing(T_INT32 port_desc, T_CONST_ADR _adr_msg, T_UINT32 size_msg, E_BOOL* _saturation, T_INT32* _exec_status);
extern E_OK_KO HAL__ReadInPortQueuing(T_INT32 port_desc, T_UINT32 max_size, T_ADR _adr_msg, E_BOOL* _no_msg, T_INT32* _read_msg_size_or_status);

#ifdef __cplusplus
} // extern "C"
#endif

#endif
```

```
/**
 * @file PartitionBase.cpp
 * @date 4 oct. 2019
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 */

#include "PartitionBase.h"
#ifdef COVERAGE_BUILD
#include <ecl/sys/cov/base.h>
#endif
#include <ecl/ExtensionPoint.h>

namespace ecl
{
namespace sys
{
namespace xm
{

PartitionBase::~PartitionBase() = default;

void PartitionBase::run()
{
    Scheduler& scheduler = ExtensionPoint<Scheduler>::impl();

    scheduler.start([this](Scheduler::SlotId slotId){ this->runSlot(slotId); });

    runBackgroundInit();

    while(not m_askedHalt)
    {
        runBackgroundLoop();
    }

#ifdef COVERAGE_BUILD
    gcov_flush();
#endif
    // when background processing is finished, wait for a call to halt.
    scheduler.waitNextSlot();
}

void PartitionBase::runBackgroundInit()
{
}

void PartitionBase::runBackgroundLoop()
{
}

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */
```

```
/**
 * @file PartitionBase.h
 * @date 4 oct. 2019
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 */

#ifndef ECL_SYS_XM_PARTITIONBASE_H_
#define ECL_SYS_XM_PARTITIONBASE_H_

#include <ecl/sys/Scheduler.h>
#include <ecl/sys/Process.h>

namespace ecl
{
namespace sys
{
namespace xm
{

class PartitionBase : public sys::Process
{
public:
    PartitionBase();
    virtual ~PartitionBase() override;

    void run() final override;
    void halt();

protected:
    virtual void runBackgroundInit();
    virtual void runBackgroundLoop();
    virtual void runSlot(Scheduler::SlotId slotId) = 0;

    bool hasAskedHalt() const;

private:
    bool m_askedHalt = false;
};

// =====
// PartitionBase inline methods bodies
// -----

inline PartitionBase::PartitionBase()
{
    // @TODO: find a way to create generic markers like:
    // MARK__WCET__IMT__START__INIT(0, 0);
}

inline void PartitionBase::halt()
{
    m_askedHalt = true;
}

inline bool PartitionBase::hasAskedHalt() const
{
    return m_askedHalt;
}

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */

#endif /* ECL_SYS_XM_PARTITIONBASE_H_ */
```

```

/**
 * @file Port.cpp
 * @date 16 juil. 2018
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 *
 */

#include "Port.h"
#include "hal.h"
#include <cassert>

// #define LOG_MODULE_SUFFIX ECL_SYS_XM_PORT
// #define ECL_LOGLEVEL_ECL_SYS_XM_PORT 1
#include <ecl/Logging.h>

namespace ecl
{
namespace sys
{
namespace xm
{
Port::Direction direction(const HAL__QUEUEING_PORT_INFO& info)
{
    return info.direction == HAL__C_PORT_DIRECTION_SOURCE ? Port::Direction::TX : Port::Direction::RX;
}

Port::Direction direction(const HAL__SAMPLING_PORT_INFO& info)
{
    return info.direction == HAL__C_PORT_DIRECTION_SOURCE ? Port::Direction::TX : Port::Direction::RX;
}

// =====
// Port class methods bodies
// =====

Port::Descriptor Port::createQueueingPort(const char8_t* portName, Id portId)
{
    HAL__QUEUEING_PORT_INFO info = {0, 0, HAL__C_PORT_DIRECTION_SOURCE};
    HAL__GetQueueingPortInfo(portName, &info);

    PLOG_NOTE1("[Port] Q(%s)", portName);

    T_INT32 halDescriptor = -1;
    E_OK_KO status = HAL__CreatePortQueueing(portName,
                                             info.maxMsgSize,
                                             info.direction,
                                             info.maxNoMsgs,
                                             &halDescriptor);

    if(not(status == OK and halDescriptor >= 0))
    {
        PLOG_ERROR("[Port] Q(%s) create fail (%d)", portName, halDescriptor);
        ExtensionPoint<FailureHandler>::impl().reportAccessFailure(portId, direction(info));
    }
    assert(status == OK and halDescriptor >= 0);

    return Descriptor(status == OK ? halDescriptor : -1);
}

Port::Descriptor Port::createSamplingPort(const char8_t* portName, Id portId)
{
    HAL__SAMPLING_PORT_INFO info = {0, 0, HAL__C_PORT_DIRECTION_SOURCE};
    HAL__GetSamplingPortInfo(portName, &info);

    PLOG_NOTE1("[Port] S(%s)", portName);

    T_INT32 halDescriptor = -1;
    E_OK_KO status = HAL__CreatePortSampling(portName,
                                             info.maxMsgSize,
                                             info.direction,
                                             info.refreshPeriod,
                                             &halDescriptor);

    if(not(status == OK and halDescriptor >= 0))
    {
        PLOG_ERROR("[Port] S(%s) create fail (%d)", portName, halDescriptor);
        ExtensionPoint<FailureHandler>::impl().reportAccessFailure(portId, direction(info));
    }
    assert(status == OK and halDescriptor >= 0);
    return Descriptor(status == OK ? halDescriptor : -1);
}

// =====
// Port::QueueingTx class methods bodies
// =====

Port::QueueingTx::QueueingTx(const char8_t* portName)

```

```

    : Port(),
      m_descriptor(createQueueingPort(portName, id())),
      m_portName(portName)
{
}

Port::Result Port::QueueingTx::receive(const MutableBuffer& /*buffer*/)
{
    assert(false);
    return {0, STATUS_FAILED};
}

Port::Result Port::QueueingTx::send(const ConstBuffer& buffer)
{
    E_BOOL isSaturated = FALSE;
    T_INT32 xmStatus = 0;
    E_OK_KO result = HAL_WriteInPortQueueing(m_descriptor,
                                              buffer.data,
                                              buffer.maxBytes,
                                              &isSaturated,
                                              &xmStatus);

    if(result == OK)
    {
        assert(xmStatus >= 0);
        return {buffer.maxBytes, STATUS_OK};
    }
    else
    {
        assert(xmStatus >= 0);
        if(isSaturated == FALSE)
        {
            PLOG_ERROR("[Port] Q(%s) TX fail (%d)", m_portName, xmStatus);
            ExtensionPoint<FailureHandler>::impl().reportAccessFailure(id(), Direction::TX);
        }
        return {0, isSaturated == TRUE ? STATUS_SATURATED : xmStatus};
    }
}

Port::Stats Port::QueueingTx::getStats() const
{
    // FIXME: proper implementation, factorized into a base class
    return {0, 0};
}

// =====
// Port::QueueingRx class methods bodies
// -----

Port::QueueingRx::QueueingRx(const char8_t* portName)
: Port(),
  m_descriptor(createQueueingPort(portName, id())),
  m_portName(portName)
{
}

Port::Result Port::QueueingRx::receive(const MutableBuffer& buffer)
{
    E_BOOL isEmpty = FALSE;
    T_INT32 statusOrByteCount = 0;
    E_OK_KO result = HAL_ReadInPortQueueing(m_descriptor,
                                              buffer.maxBytes,
                                              buffer.data,
                                              &isEmpty,
                                              &statusOrByteCount);

    if(result == OK)
    {
        if(isEmpty == FALSE)
        {
            assert(statusOrByteCount >= 0); // when read succeeded, statusOrByteCount is positive.
            return {static_cast<ByteCount>(statusOrByteCount), STATUS_OK};
        }
        else
        {
            return {0, STATUS_EMPTY};
        }
    }
    else
    {
        assert(statusOrByteCount < 0); // when read failed, statusOrByteCount is negative.
        PLOG_ERROR("[Port] Q(%s) RX fail (%d)", m_portName, statusOrByteCount);
        ExtensionPoint<FailureHandler>::impl().reportAccessFailure(id(), Direction::RX);
        return {0, statusOrByteCount};
    }
}

Port::Result Port::QueueingRx::send(const ConstBuffer& /*buffer*/)
{
    assert(false);
    return {0, STATUS_FAILED};
}

Port::Stats Port::QueueingRx::getStats() const
{
}

```

3

3

```
// FIXME: proper implementation, factorized into a base class
return {0, 0};
```

}

```
// =====
// Port::QueuingRx class methods bodies
// =====
```

```
Port::QueuingDuplex::QueuingDuplex(const char8_t* rxPortName, const char8_t* txPortName)
    : Port(),
      m_rxPort(rxPortName),
      m_txPort(txPortName)
{
}
```

```
Port::Result Port::QueueingDuplex::receive(const MutableBuffer& buffer)
{
    return m_rxPort.receive(buffer);
}
```

```
Port::Result Port::QueuingDuplex::send(const ConstBuffer& buffer)
{
    return m_txPort.send(buffer);
}
```

```
Port::Stats Port::QueueingDuplex::getStats() const
{
    // FIXME: proper implementation, factorized into a base class
    return {0, 0};
}
```

```

// =====
// Port::SamplingTx class methods bodies
// =====

```

```
Port::SamplingTx::SamplingTx(const char8_t* portName)
    : Port(),
      m_descriptor(createSamplingPort(portName, id())),
      m_portName(portName)
{
}
```

```
Port::Result Port::SamplingTx::receive(const MutableBuffer& /*buffer*/)
{
    assert(false);
    return {0, STATUS_FAILED};
}
```

```
Port::Result Port::SamplingTx::send(const ConstBuffer& buffer)
{
    T_INT32 xmStatus = 0;
    E_OK_KO result = HAL__WriteInPortSampling(m_descriptor,
                                              buffer.data,
                                              buffer.maxBytes,
                                              &xmStatus);

    if(result == OK)
    {
        assert(xmStatus >= 0);
        return {buffer.maxBytes, STATUS_OK};
    }
    else
    {
        assert(xmStatus < 0);
        PLOG_ERROR("[Port] S(%s) TX fail (%d)", m_portName, xmStatus);
        ExtensionPoint<FailureHandler>::impl().reportAccessFailure(id(), Direction::TX);
        return {0, xmStatus};
    }
}
```

```
Port::Stats Port::SamplingTx::getStats() const
{
    // FIXME: proper implementation, factorized into a base class
    return {0, 0};
}
```

```

// =====
// Port::SamplingRx class methods bodies
// =====

```

```
Port::SamplingRx::SamplingRx(const char8_t* portName)
    : Port(),
      m_descriptor(createSamplingPort(portName, id())),
      m_portName(portName)
{
}
```

[illegible]

```
        buffer.data,
        &isValid,
        &isEmpty,
        &statusOrByteCount);

    if(result == OK)
    {
        if(isEmpty == TRUE)
        {
            assert(statusOrByteCount == -1); // expect following XM error code: XM_NO_ACTION (-1)
            return {0, STATUS_EMPTY};
        }
        else
        {
            // received bytes should not exceed buffer size
            assert(statusOrByteCount <= static_cast<T_INT32>(buffer.maxBytes));

            // message is present, even if too old
            const Status status = (isValid == TRUE) ? STATUS_OK : STATUS_TOO_OLD;

            return {static_cast<ByteCount>(statusOrByteCount), status};
        }
    }
    else
    {
        // expect following XM error codes
        // XM_INVALID_PARAM: -3
        // XM_INVALID_CONFIG: -5
        assert(statusOrByteCount == -3 or statusOrByteCount == -5);
        PLOG_ERROR("[Port] S(%s) RX fail (%d)", m_portName, statusOrByteCount);
        ExtensionPoint<FailureHandler>::impl().reportAccessFailure(id(), Direction::RX);
        return {0, STATUS_FAILED};
    }
}

Port::Result Port::SamplingRx::send(const ConstBuffer& /*buffer*/)
{
    assert(false);
    return {0, STATUS_FAILED};
}

Port::Stats Port::SamplingRx::getStats() const
{
    // FIXME: proper implementation, factorized into a base class
    return {0, 0};
}

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */
```

```

/**
 * @file Port.h
 * @date 16 juil. 2018
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat à l'Energie Atomique et aux Energies Alternatives (CEA)
 */

#ifndef ECL_SYS_XM_PORT_H_
#define ECL_SYS_XM_PORT_H_

#include <ecl/sys/Port.h>
#include <ecl/core/cc/Types.h>
#include <ecl/Events.h>

namespace ecl
{
    namespace sys
    {
        namespace xm
        {
            class Port
            {
            public:
                using Descriptor = int32_t;
                using Result = sys::Port::Result;
                using Status = sys::Port::Status;
                using Stats = sys::Port::Stats;
                using Id = sys::Port::Id;
                using Direction = sys::Port::Direction;
                using FailureHandler = sys::Port::FailureHandler;
                using PartitionId = core::PartitionId;

                class QueuingTx;
                class QueuingRx;
                class QueuingDuplex;

                class SamplingTx;
                class SamplingRx;

                template<PartitionId PARTITION, Events EID_ACCESS_FAILURE>
                class FailureReporter;

                static constexpr Status STATUS_TOO_OLD = 2;

            private:
                static Descriptor createQueuingPort(const char8_t* portName, Id portId);
                static Descriptor createSamplingPort(const char8_t* portName, Id portId);
            };

            // =====
            // Port inner classes declarations
            // =====

            class Port::QueuingTx : public sys::Port
            {
            public:
                QueuingTx(const char8_t* portName);
                ~QueuingTx() override = default;

                Result receive(const MutableBuffer& buffer) override;
                Result send(const ConstBuffer& buffer) override;

                Stats getStats() const override;

            private:
                Descriptor m_descriptor;
                const char8_t* const m_portName;
            };

            class Port::QueuingRx final : public sys::Port
            {
            public:
                QueuingRx(const char8_t* portName);
                ~QueuingRx() override = default;

                Result receive(const MutableBuffer& buffer) override;
                Result send(const ConstBuffer& buffer) override;

                Stats getStats() const override;

            private:
                Descriptor m_descriptor;
                const char8_t* const m_portName;
            };
        }
    }
}

```

```

class Port::QueuingDuplex final : public sys::Port
{
public:
    QueuingDuplex(const char8_t* rxPortName, const char8_t* txPortName);
    ~QueuingDuplex() override = default;

    Result receive(const MutableBuffer& buffer) override;
    Result send(const ConstBuffer& buffer) override;

    Stats getStats() const override;

private:
    QueuingRx m_rxPort;
    QueuingTx m_txPort;
};

class Port::SamplingTx final : public sys::Port
{
public:
    SamplingTx(const char8_t* portName);
    ~SamplingTx() override = default;

    Result receive(const MutableBuffer& buffer) override;
    Result send(const ConstBuffer& buffer) override;

    Stats getStats() const override;

private:
    Descriptor m_descriptor;
    const char8_t* const m_portName;
};

class Port::SamplingRx final : public sys::Port
{
public:
    SamplingRx(const char8_t* portName);
    ~SamplingRx() override = default;

    Result receive(const MutableBuffer& buffer) override;
    Result send(const ConstBuffer& buffer) override;

    Stats getStats() const override;

private:
    Descriptor m_descriptor;
    const char8_t* const m_portName;
};

template<Port::PartitionId PARTITION, Events EID_ACCESS_FAILURE>
class Port::FailureReporter : public sys::Port::FailureHandler
{
public:
    FailureReporter() = default;
    ~FailureReporter() override = default;

    void reportAccessFailure(Id portId, Direction portDirection) override;
};

// =====
// Port::ErrorReporter class methods bodies
// =====

template<Port::PartitionId PARTITION, Events IEID_ACCESS_FAILURE>
void Port::FailureReporter<PARTITION, IEID_ACCESS_FAILURE>::reportAccessFailure(Id portId, Direction portDirection)
{
    // UNKNOWN because there is no way to determine the partition we are communicating with
    // other than specify it when creating the port, with no guarantee it is correct.
    PartitionId srcPart = (portDirection == Direction::TX ? PARTITION : PartitionId::UNKNOWN);
    PartitionId dstPart = (portDirection == Direction::RX ? PARTITION : PartitionId::UNKNOWN);
    EventObsw::report<IEID_ACCESS_FAILURE>(srcPart, dstPart, portId);
}

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */

#endif /* ECL_SYS_XM_PORT_H_ */

```

```
/**
 * @file Scheduler.cpp
 * @date 25 juil. 2019
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 */

#include <ecl/ExtensionPoint.h>
#include <ecl/sys/gr712/CpuServices.h>
#include <ecl/sys/xm/Scheduler.h>
#include <ecl/sys/xm/hal.h>
#include <ecl/ExtensionPoint.h>

// #define LOG_MODULE_SUFFIX ECL_SYS_XM_SCHEDULER
// #define ECL_LOGLEVEL ECL_SYS_XM_SCHEDULER 10
#include <ecl/Logging.h>

namespace ecl
{
    namespace sys
    {
        namespace xm
        {

Scheduler::Scheduler()
    : m_slotHandler(),
      m_handlerStartTime(SLOT_NO_START),
      m_handlerSlotId(SLOT_NONE),
      m_lastSlotId(SLOT_NONE),
      m_backgroundBeginTime(SLOT_NO_START),
      m_backgroundBeginSlotId(SLOT_NONE),
      m_inHandler(true), // consider being in handler before execution of the first slot handler
      m_wasOverrun(false),
      m_lastHasOverrun(false)
{
    PLOG_DEBUG1("[SCD] ctor");
}

Scheduler::~Scheduler()
{
    PLOG_DEBUG1("[SCD] dtor");
}

void Scheduler::start(SlotHandler slotHandler)
{
    m_slotHandler = slotHandler;
    auto& sysSvc = ExtensionPoint<sys::SystemServices>::impl();
    sysSvc.softIrqSubscribe(SoftIrqNumber::SLOT_START, slotHandlerProxy);
    sysSvc.softIrqAllow(SoftIrqNumber::SLOT_START);
    sysSvc.irqsEnable();
}

void Scheduler::stop()
{
    auto& sysSvc = ExtensionPoint<sys::SystemServices>::impl();
    sysSvc.irqsDisable();
    sysSvc.softIrqSubscribe(SoftIrqNumber::SLOT_START, nullptr);
}

Scheduler::SlotId Scheduler::getCurrentSlotId()
{
    return HAL__GetSlotId();
}

Scheduler::SlotId Scheduler::getHandlerSlotId()
{
    return m_handlerSlotId;
}

Scheduler::TimePoint Scheduler::getHandlerStartTime()
{
    return m_handlerStartTime;
}

bool Scheduler::wasOverrun()
{
    return m_wasOverrun;
}

void Scheduler::executeSlotHandler()
{
    if(m_handlerSlotId == m_lastSlotId) // previous handler finished its execution
    {
        // WARNING: in case of overrun, the handler will be not started at the beginning of the slot!
        // So, m_handlerStartTime cannot be used to know the remaining time in the slot.
        m_handlerStartTime = CpuMainClock::now();
    }
}
```

```

    m_handlerSlotId = getCurrentSlotId();
    m_inHandler = true;

    m_wasOverrun = m_lastHasOverrun;
    m_lastHasOverrun = false;

    m_slotHandler(m_handlerSlotId);

    SlotId currentSlotId = getCurrentSlotId();
    if(currentSlotId != m_handlerSlotId)
    {
        m_lastHasOverrun = true;
        PLOG_ERROR("Slot overrun %lu->%lu", m_handlerSlotId, currentSlotId);

        // should send event report
        ExtensionPoint<ErrorHandler>::impl()
            .reportOverrun(m_handlerSlotId, currentSlotId, sys::CpuMainClock::now() - m_handlerStartTime);
    }
    PLOG_DEBUG1("S[%lu] %lld", m_handlerSlotId, (sys::CpuMainClock::now() - m_handlerStartTime).count());

    m_lastSlotId = m_handlerSlotId;
    m_inHandler = false;
}
else
{
    PLOG_ERROR("Handler exec rejected!");
}
}

bool Scheduler::inHandler()
{
    return m_inHandler;
}

void Scheduler::waitNextSlot()
{
    PLOG_DEBUG1("BG(%lu,%lu) %lld", m_backgroundBeginSlotId, getCurrentSlotId(),
        (sys::CpuMainClock::now() - m_backgroundBeginTime).count());
    m_backgroundBeginSlotId = sys::Scheduler::SLOT_NONE;

    HAL__WaitNextSlot();

    m_backgroundBeginTime = sys::CpuMainClock::now();
    m_backgroundBeginSlotId = getCurrentSlotId();
}

void Scheduler::wait(TimeDuration duration)
{
    #ifdef NDEBUG
        static constexpr uint32_t LOOP_CYCLES = 3;
        static constexpr uint32_t OVERHEAD_CYCLES = 25;
    #else
        static constexpr uint32_t LOOP_CYCLES = 15;
        static constexpr uint32_t OVERHEAD_CYCLES = 1600;
    #endif

    static constexpr uint32_t NOP_CYCLES = 8;
    static constexpr uint32_t ITER_CYCLES = NOP_CYCLES + LOOP_CYCLES;

    const uint32_t spinCount = getCycles(duration) / ITER_CYCLES;
    for(uint32_t i = OVERHEAD_CYCLES / ITER_CYCLES; i < spinCount; ++i)
    {
        asm volatile ("nop"); asm volatile ("nop");
        asm volatile ("nop"); asm volatile ("nop");
        asm volatile ("nop"); asm volatile ("nop");
        asm volatile ("nop"); asm volatile ("nop");
    }
}

void Scheduler::slotHandlerProxy(SystemServices::TrapContext*)
{
    ExtensionPoint<sys::Scheduler>::impl().executeSlotHandler();
}

uint64_t Scheduler::getCycles(TimeDuration duration)
{
    return duration * uint32_t(gr712::CpuServices::CPU_FREQUENCY) / std::chrono::seconds(1);
}

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */

```

```
/**
 * @file Scheduler.h
 * @date 25 juil. 2019
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 */

#ifndef ECL_SYS_XM_SCHEDULER_H_
#define ECL_SYS_XM_SCHEDULER_H_

#include <ecl/sys/Scheduler.h>
#include <ecl/sys/SystemServices.h>
#include <ecl/Events.h>

namespace ecl
{
    namespace sys
    {
        namespace xm
        {

class Scheduler final : public sys::Scheduler
{
public:
    template<Events IEID_OVERRUN>
    class OverrunErrorHandler;

    Scheduler();
    ~Scheduler() override;

    void start(SlotHandler slotHandler) override;
    void stop() override;

    void executeSlotHandler() override;

    SlotId getCurrentSlotId() override;
    SlotId getHandlerSlotId() override;
    TimePoint getHandlerStartTime() override;
    bool wasOverrun() override;
    bool inHandler() override;

    void waitNextSlot() override;
    void wait(TimeDuration duration) override;

private:
    static void slotHandlerProxy(SystemServices::TrapContext*);
    static uint64_t getCycles(TimeDuration duration);

    SlotHandler m_slotHandler;
    TimePoint m_handlerStartTime;
    SlotId m_handlerSlotId;
    SlotId m_lastSlotId;
    TimePoint m_backgroundBeginTime;
    SlotId m_backgroundBeginSlotId;
    bool m_inHandler;
    bool m_wasOverrun;
    bool m_lastHasOverrun;
};

template<Events IEID_OVERRUN>
class Scheduler::OverrunErrorHandler final : public sys::Scheduler::ErrorHandler
{
public:
    OverrunErrorHandler() = default;
    ~OverrunErrorHandler() override = default;

    void reportOverrun(SlotId startSlotId, SlotId finishSlotId, TimeDuration duration) override;
};

template<Events IEID_OVERRUN>
void Scheduler::OverrunErrorHandler<IEID_OVERRUN>::reportOverrun(SlotId startSlotId, SlotId finishSlotId, TimeDuration duration)
{
    EventObsw::report<IEID_OVERRUN>(uint8_t(startSlotId), uint8_t(finishSlotId), duration.count());
}

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */

#endif /* ECL_SYS_XM_SCHEDULER_H_ */
```

```
/**
 * @file SystemServices.cpp
 * @date 25 juil. 2019
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 *
 */

#include "StandaloneSystemServices.h"
#include "hal.h"
#include <ecl/Enum.h>
#include <ecl/Logging.h>

namespace ecl
{
namespace sys
{
namespace xm
{

StandaloneSystemServices::StandaloneSystemServices()
{

}

StandaloneSystemServices::~StandaloneSystemServices() = default;

void StandaloneSystemServices::softIrqSubscribe(SoftIrqNumber irqNumber, TrapHandler handler)
{
    (void) irqNumber;
    (void) handler;
}

void StandaloneSystemServices::hardIrqSubscribe(HardIrqNumber irqNumber, TrapHandler handler)
{
    (void) irqNumber;
    (void) handler;
}

void StandaloneSystemServices::softIrqAllow(SoftIrqNumber irqNumber)
{
    (void) irqNumber;
}

void StandaloneSystemServices::hardIrqAllow(HardIrqNumber irqNumber)
{
    (void) irqNumber;
}

void StandaloneSystemServices::irqsEnable()
{
}

void StandaloneSystemServices::irqsDisable()
{
}

void StandaloneSystemServices::unexpectedTrapHandler(TrapContext* trapCtxt)
{
    (void) trapCtxt;
}

CpuMainClock::time_point StandaloneSystemServices::currentTime() const
{
    T_UINT64 timeUs = static_cast<T_UINT64>(std::clock());

    return CpuMainClock::time_point(std::chrono::microseconds((timeUs/CLOCKS_PER_SEC)*1000));
}

CpuMainClock::time_point StandaloneSystemServices::lastPpsTime() const
{
    T_UINT64 microsecsSinceEpoch = 0;
    return CpuMainClock::time_point(std::chrono::microseconds(microsecsSinceEpoch));
}

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */
```

```
/**
 * @file StandaloneSystemServices.h
 * @date 6 juil. 2020
 * @author pkestene
 * @note this is a dummy, i.e. almost empty SystemServices implementation
 *       for standalone usege (no Xtratum) that allow to instantiate, e.g. a
 *       Uart::Port
 *
 * Contributors: pkestene
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 *
 */

#ifndef ECL_SYS_XM_STANDALONESYSTEMSERVICES_H_
#define ECL_SYS_XM_STANDALONESYSTEMSERVICES_H_

#include <ecl/sys/SystemServices.h>

namespace ecl
{
    namespace sys
    {
        namespace xm
        {

class StandaloneSystemServices final : public sys::SystemServices
{
public:
    StandaloneSystemServices();
    ~StandaloneSystemServices() override;

    void softIrqSubscribe(SoftIrqNumber irqNumber, TrapHandler handler) override;
    void hardIrqSubscribe(HardIrqNumber irqNumber, TrapHandler handler) override;
    void softIrqAllow(SoftIrqNumber irqNumber) override;
    void hardIrqAllow(HardIrqNumber irqNumber) override;
    void irqsEnable() override;
    void irqsDisable() override;

    CpuMainClock::time_point currentTime() const override;

    CpuMainClock::time_point lastPpsTime() const override;

private:
    static void unexpectedTrapHandler(TrapContext* trapCtxt);
};

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */

#endif /* ECL_SYS_XM_STANDALONESYSTEMSERVICES_H_ */
```

```

/**
 * @file SystemServices.cpp
 * @date 25 juil. 2019
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 *
 */

#include "SystemServices.h"
#include "hal.h"
#include <ecl/Enum.h>

// #define LOG_MODULE_SUFFIX ECL_SYS_XM_SYSTEMSERVICES
// #define ECL_LOGLEVEL_ECL_SYS_XM_SYSTEMSERVICES 5
#include <ecl/Logging.h>

namespace ecl
{
namespace sys
{
namespace xm
{

static_assert(underlyingValue(SoftIrqNumber::SHUTDOWN) == HAL__C_SOFT_IRQ_ID_SHUTDOWN,
    "Signal::SHUTDOWN definition mismatch");
static_assert(underlyingValue(SoftIrqNumber::SLOT_START) == HAL__C_SOFT_IRQ_ID_SLOT_START,
    "Signal::SLOT_START definition mismatch");
static_assert(underlyingValue(SoftIrqNumber::TIMER) == HAL__C_SOFT_IRQ_ID_TIMER,
    "Signal::TIMER definition mismatch");
static_assert(sizeof(SystemServices::TrapContext) == 12 * 4,
    "TrapContext definition mismatch");

SystemServices::SystemServices()
{
    PLOG_NOTE1("[%u] [SysSvc] ctor", HAL__GetPartitionId());
    HAL__Init(reinterpret_cast<HAL__T_HYPERVISOR_TRAP_HANDLER>(unexpectedTrapHandler));
}

SystemServices::~SystemServices() = default;

void SystemServices::softIrqSubscribe(SoftIrqNumber irqNumber, TrapHandler handler)
{
    HAL__SoftIrqSubscribe(
        static_cast<HAL__E_SOFT_IRQ_ID>(irqNumber),
        reinterpret_cast<HAL__T_HYPERVISOR_TRAP_HANDLER>(handler));
}

void SystemServices::hardIrqSubscribe(HardIrqNumber irqNumber, TrapHandler handler)
{
    HAL__HardIrqSubscribe(
        static_cast<HAL__E_HARD_IRQ_ID>(irqNumber),
        reinterpret_cast<HAL__T_HYPERVISOR_TRAP_HANDLER>(handler));
}

void SystemServices::softIrqAllow(SoftIrqNumber irqNumber)
{
    HAL__SoftIrqAllow(static_cast<HAL__E_SOFT_IRQ_ID>(irqNumber));
}

void SystemServices::hardIrqAllow(HardIrqNumber irqNumber)
{
    HAL__HardIrqAllow(static_cast<HAL__E_HARD_IRQ_ID>(irqNumber));
}

void SystemServices::irqsEnable()
{
    HAL__IrqsEnable();
}

void SystemServices::irqsDisable()
{
    HAL__IrqsDisable();
}

void SystemServices::unexpectedTrapHandler(TrapContext* trapCtxt)
{
    (void)trapCtxt; // prevent "unused" in case logging is disabled.
    PLOG_ERROR("Unexpected trap! irq=%lu pc=%lx", trapCtxt->irqNr, trapCtxt->pc);
}

CpuMainClock::time_point SystemServices::currentTime() const
{
    T_UINT64 timeUs;
    HAL__GetTimeUs(&timeUs);
    return CpuMainClock::time_point(std::chrono::microseconds(timeUs));
}

```

```
CpuMainClock::time_point SystemServices::lastPpsTime() const
{
    T_UINT64 microsecsSinceEpoch = HAL__GetIrqLastOccurenceTime(underlyingValue(HardIrqNumber::PPS));
    return CpuMainClock::time_point(std::chrono::microseconds(microsecsSinceEpoch));
}

CpuMainClock::time_point SystemServices::lastTfTime() const
{
    T_UINT64 microsecsSinceEpoch = HAL__GetIrqLastOccurenceTime(underlyingValue(HardIrqNumber::TF));
    return CpuMainClock::time_point(std::chrono::microseconds(microsecsSinceEpoch));
}

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */
```

```
/**
 * @file SystemServices.h
 * @date 25 juil. 2019
 * @author fchateau
 * @note
 *
 * Contributors: fchateau
 *
 * This file is part of the UGTS software project.
 *
 * @copyright Â© Commissariat a l'Energie Atomique et aux Energies Alternatives (CEA)
 *
 */

#ifndef ECL_SYS_XM_SYSTEMSERVICES_H_
#define ECL_SYS_XM_SYSTEMSERVICES_H_

#include <ecl/sys/SystemServices.h>

namespace ecl
{
    namespace sys
    {
        namespace xm
        {

class SystemServices final : public sys::SystemServices
{
public:
    SystemServices();
    ~SystemServices() override;

    void softIrqSubscribe(SoftIrqNumber irqNumber, TrapHandler handler) override;
    void hardIrqSubscribe(HardIrqNumber irqNumber, TrapHandler handler) override;
    void softIrqAllow(SoftIrqNumber irqNumber) override;
    void hardIrqAllow(HardIrqNumber irqNumber) override;
    void irqsEnable() override;
    void irqsDisable() override;

    CpuMainClock::time_point currentTime() const override;

    CpuMainClock::time_point lastPpsTime() const override;
    CpuMainClock::time_point lastTfTime() const override;

private:
    static void unexpectedTrapHandler(TrapContext* trapCtxt);
};

} /* namespace xm */
} /* namespace sys */
} /* namespace ecl */

#endif /* ECL_SYS_XM_SYSTEMSERVICES_H_ */
```